

Method Resolution Order

```
class A:
    def process(self):
        print("A process()")
```

```
class B:
    pass
```

```
class C(A, B):
    pass
```

```
obj = C()
```

```
C.mro()
```

```
[__main__.C, __main__.A, __main__.B, object]
```

```
obj.process()
```

```
A process()
```

```
class B:
    def process(self):
        print("B process()")
```

```
class C(A, B):
    pass
```

```
obj = C()
obj.process()
C.mro()
```

```
A process()
[__main__.C, __main__.A, __main__.B, object]
```

```
class C(B, A):
    pass
```

```
obj = C()
obj.process()
C.mro()
```

```
B process()
[__main__.C, __main__.B, __main__.A, object]
```

```
class C(A, B):
    def process(self):
        print("C process()")
```

```
class D(C,B):
    pass
```

```
obj = D()
obj.process()
D.mro()
```

```
C process()
[__main__.D, __main__.C, __main__.A, __main__.B, object]
```

```
class A:
    def process(self):
        print("A process()")
```

```
class B(A):
    pass
class C(A):
    def process(self):
        print("C process()")
class D(B,C):
    pass
```

```
obj = D()
obj.process()
D.mro()
```

```
C process()
[__main__.D, __main__.B, __main__.C, __main__.A, object]
```

```
class A:
    def process(self):
        print("A process()")
class B(A):
    def process(self):
        print("B process()")
class C(A, B): # Here A and B are same thing : Error
    pass
```

```
-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_8627/479812539.py in <cell line: 0>()
      5     def process(self):
      6         print("B process()")
----> 7 class C(A, B): # Here A and B are same thing : Error
      8     pass
```

```
TypeError: Cannot create a consistent method resolution
order (MRO) for bases A, B
```

Start coding or [generate](#) with AI.